Appendix: Before-and-After Build Trees

This appendix shows what existed before each build, what the build added, why each output is included, and common viva questions.

A. Step 1 — OP-TEE

```
BEFORE: optee_os/
■■■ core/ — secure-kernel source
■■■ lib/ — secure APIs
■■■ mk/ — build rules
■■■ ta/ — TA source

AFTER: optee_os/out/arm/ NEW
■■■ core/tee.bin — complete Secure World binary
■■■ core/tee.elf — debug/symbol image
■■■ core/tee-header_v2.bin — TF-A loading header
■■■ core/tee-pager_v2.bin — main secure payload
■■■ export-ta_arm64/ — TA development files
```

Question	Deep answer
Why include OP-TEE?	Linux is not trusted for secret keys and final security decisions; OP-TEE provides Secure World execution.
Why several tee files?	The selected TF-A format uses header/pager/pageable pieces; tee.elf supports debugging.
Does tee.bin prove runtime?	No. It proves compilation only.

B. Step 2 — U-Boot and TF-A

```
U-BOOT BEFORE
u-boot/{arch, board, cmd, configs/rpi_3_defconfig, drivers, Makefile}

U-BOOT AFTER
■■■ .config — selected RPi3 configuration
■■■ u-boot — ELF/debug file
■■■ u-boot.bin — NEW raw BL33 payload
■■■ System.map — symbols
■■■ tools/ — boot-image/environment tools

TF-A BEFORE
trusted-firmware-a/{bl1, plat/rpi3, services, Makefile}

TF-A AFTER
trusted-firmware-a/build/rpi3/debug/
■■■ bl1.bin — early secure firmware
■■■ fip.bin — BL32 OP-TEE + BL33 U-Boot
■■■ armstub8.bin — padded BL1 + FIP
```

Question	Deep answer
Why both TF-A and U-Boot?	TF-A performs secure initialization/world switching; U-Boot loads Linux, the DTB and arguments.
Why BL32?	BL32 is TF-A's Secure Payload position used for OP-TEE.
Why BL33?	BL33 is the non-secure payload used for U-Boot.
Why no separate FIP copy?	The RPi3 armstub8.bin already includes the FIP after padded BL1.

C. Step 3 — Linux and Device Trees

```
BEFORE: linux/
■■■ arch/arm64/ — ARM64 source/configuration
■■■ arch/arm64/boot/dts/ — existing DTS source
■■■ drivers/ — includes TEE drivers
■■■ fs/ and net/
■■■ Makefile

AFTER: linux/
■■■ .config — generated configuration
■■■ vmlinux + System.map — debug/symbol outputs
■■■ arch/arm64/boot/Image — NEW ARM64 kernel
■■■ .../bcm2710-rpi-3-b.dtb — compiled existing DTS
■■■ .../bcm2710-rpi-3-b-plus.dtb — compiled existing DTS
■■■ generated .o and .ko files
```

Question	Deep answer
Why change Linux config?	To select ARM64/RPi3 support and include CONFIG_TEE plus CONFIG_OPTEE.
Why DTBs without new hardware?	ARM Linux needs a board description. Existing RPi3 DTS source was compiled; no customization was made.
Why two DTBs?	The project covers Model B and B+; only the matching physical-board DTB is used.
Does this prove communication?	No. Board boot and xtest/example success are required.

D. Step 4 — Verification Tree

```
BEFORE
Minor_Project/
■■■ optee_full/ — complete, not formally selected
■■■ optee_project/ — incomplete laptop copy
■■■ Step3_Output/
■■■ loose outputs

AFTER
Minor_Project/
■■■ optee_full/ — SELECTED authoritative tree
■■■ Step3_Output/ — preserved
■■■ Step4_Verification/
■■■ STEP4_COMPLETION.md
■■■ verified_artifacts/{optee, tf-a, u-boot, linux}
```

Question	Deep answer
Why backup outputs?	Later builds may fail or replace them; backup preserves known successful evidence.
Why SHA-256?	It fingerprints files to detect corruption or replacement.
Why compare revisions?	Matching versions improve compatibility and reproducibility.
Why inspect rpi3.mk?	It defines the supported dependency/packaging flow.

E. Step 5 — Buildroot

```
BEFORE
optee_full/
■■■ buildroot/ — recipes, not a root filesystem
■■■ optee_client/ — libteec + tee-supplicant source
■■■ optee_test/ — xtest + test TA source
■■■ optee_examples/ — host + TA source

CURRENT
■■■ module_output/ — about 1,713 module-related files
■■■ out-br/{build, host, per-package, incomplete target}

EXPECTED AFTER SUCCESS
out-br/
■■■ target/usr/sbin/tee-supplicant
■■■ target/usr/lib/libteec.so*
■■■ target/usr/bin/xtest
■■■ target/usr/bin/optee_example_*
■■■ target/lib/optee_armtz/*.ta
■■■ target/lib/modules/<kernel>/*.ko
■■■ images/rootfs.ext4
```

Question	Purpose/answer
Why tee-supplicant?	Normal World RPC help for TA loading and storage-file access.
Why libteec?	Standard API for opening sessions and invoking TA commands.
Why xtest?	Standard end-to-end runtime verification.
Why .ta files?	Security-sensitive code executed by OP-TEE.
Why matching modules?	Modules must correspond to the exact kernel Image.
Why rootfs.ext4?	Linux mounts it as / for commands, libraries, services and applications.
Why missing now?	The configured compiler prefix/path differs from the available AArch64 toolchain.

F. Complete Purpose Chain

Included item	Why included / what happens if missing
OP-TEE	Runs secure validation and TAs; without it there is no Secure World application environment.
TF-A	Performs secure initialization and world switching; without it the selected BL32 startup path is absent.
U-Boot	Loads Image, DTB and arguments; without it this flexible Linux-loading stage is absent.
Linux Image	Runs the Normal World OS and TEE driver.
DTB	Describes CPU, memory, buses and devices to Linux.
Kernel modules	Supply matching optional drivers/features.
Root filesystem	Supplies commands, libraries, services, configuration and applications.
libteec	Standard client session/command API.
tee-supplicant	Normal World RPC assistance.
xtest	Standard proof of the complete runtime path.
Host application	Ordinary caller that sends permitted requests.
Trusted Application	Security-sensitive project logic executed in Secure World.